

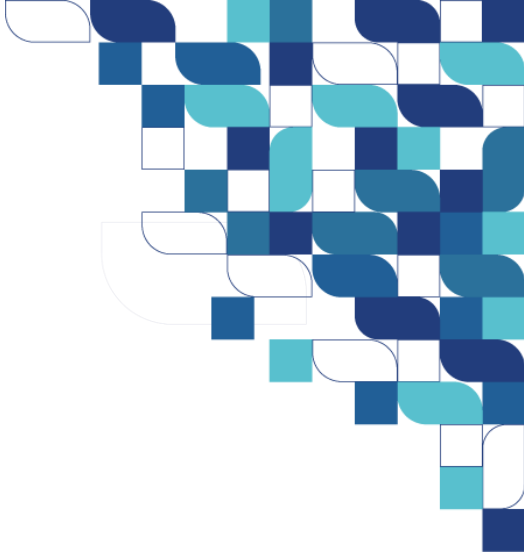
CCP230
Desenvolvimento de Algoritmos

Profa. Dra. Gabriela Oliveira Biondi

Departamento de Ciência da Computação FEI

FEI JESUÍTAS BRASIL



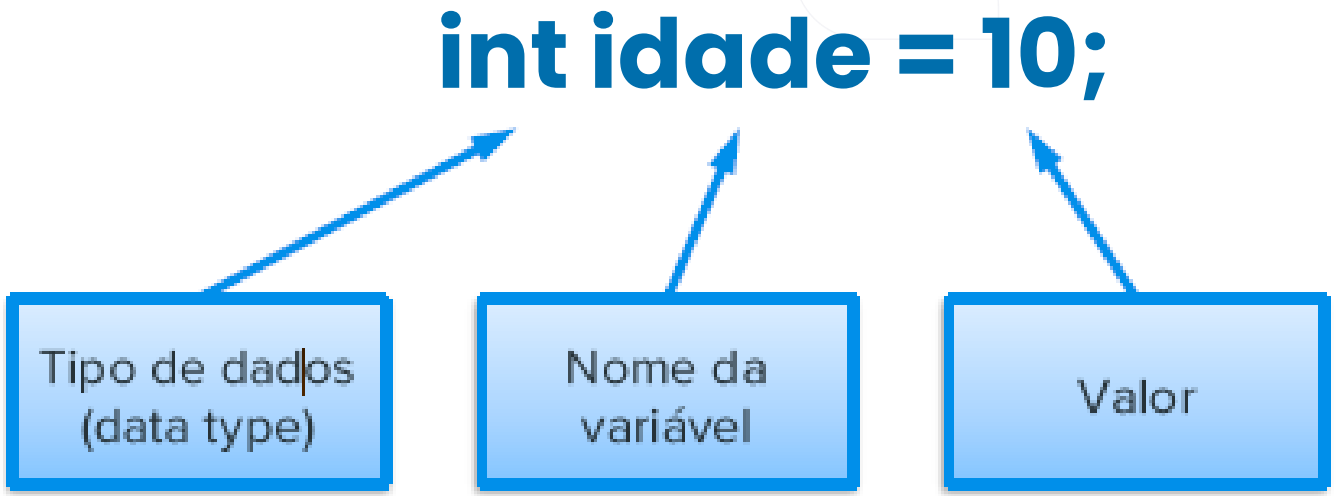


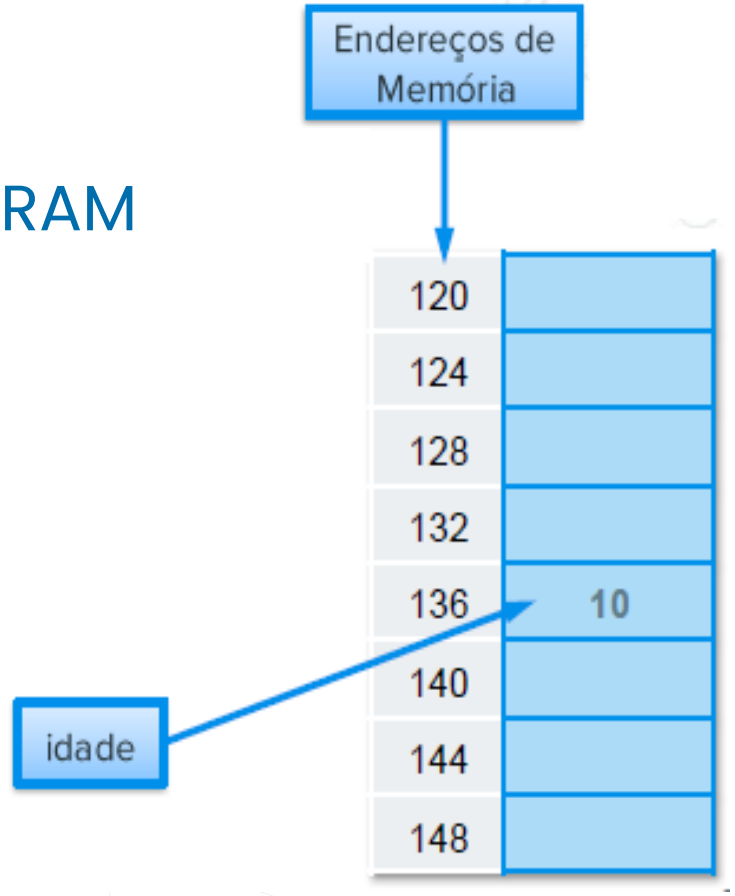
Ponteiros

Aula 09

Variáveis

- As variáveis declaradas e inicializadas nos programas são armazenadas na memória RAM
- Exemplo:



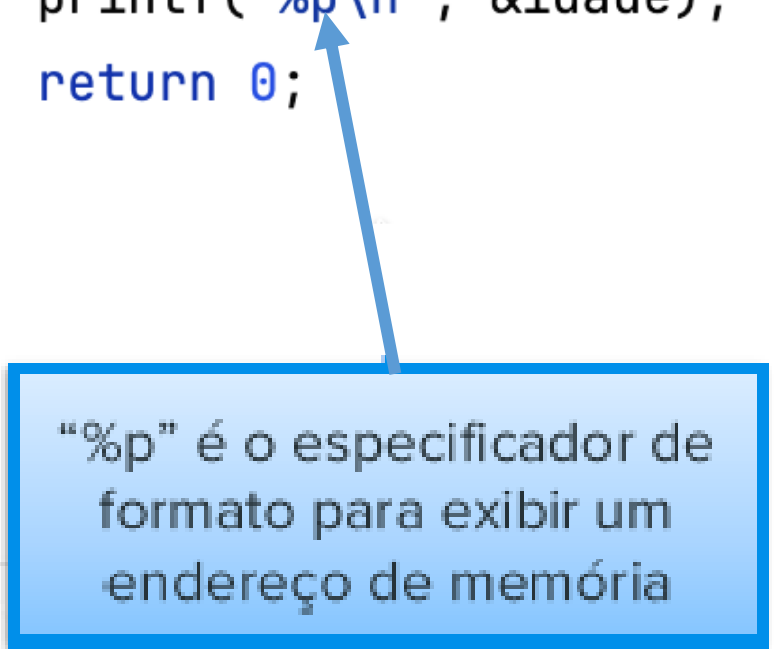


Variáveis

- Tem como eu saber em qual posição da memória minha variável está?
- Sim, por meio do operador de endereço: **&**

Variáveis

```c
#include <stdio.h>
int main() {
    int idade = 10;
    printf("%p\n", &idade);
    return 0;
}
```



Saída no terminal:

```
0x16bc97038

Process finished with exit code 0
```

Variáveis

- O **endereço** em que uma variável é armazenada é normalmente escolhido pelo computador
- Este **endereço** é dado por um **número** (quase sempre exibido em hexadecimal)
- É possível **ARMAZENAR** um endereço de uma variável em outra variável ?
- **SIM, ATRAVÉS DE PONTEIROS!**

Ponteiros

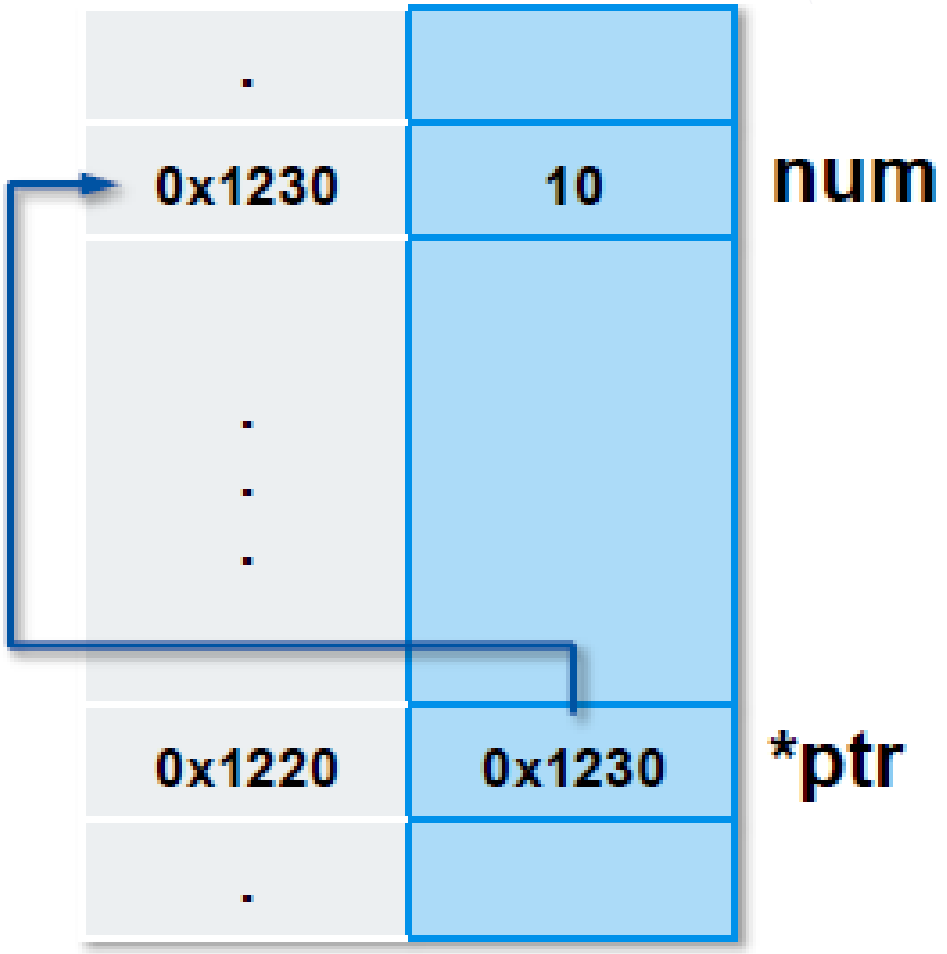
- Pode-se criar uma variável especial para armazenar um endereço:
- Essa variável é chamada de **ponteiro**
- O ponteiro contém, como valor armazenado, um **endereço de memória**

Ponteiros

- Dada uma certa variável num, um ponteiro **ptr** contém o endereço de **num**

- Assim, **ptr** aponta para **num**

Ponteiro para inteiro
(*pointer to integer*)



Ponteiros

- Na linguagem C é preciso especificar o tipo da variável “apontada”
- Sintaxe:

<tipo> *nome_variavel;

- Exemplo:

int *x_ponteiro;

Ponteiros

- O operador de ponteiro * é o complemento do operador **&**
- * é um operador unário que devolve o valor da variável localizada no endereço que o segue
- O operador * pode ser imaginado como “conteúdo de” ou “apontado por”
- **Cuidado:** Um ponteiro deve sempre apontar para um local válido antes de ser utilizado!

Ponteiros

- Exemplo em linguagem C: Ponteiro de inteiros

```c
#include <stdio.h>
int main(void) {
    int a = 10; // Declaração e inicialização de "a"
    int *aPtr;  // Declaração do ponteiro para inteiro
    aPtr = &a;  // Atribuição do endereço de "a" para "aPtr"

    printf("Conteúdo de a: %d \n", a);
    printf("Conteúdo de aPtr: %p \n", aPtr);
    printf("Endereço de a: %p \n", &a);
    printf("Endereço de aPtr: %p \n", &aPtr);
    printf("Valor apontado por aPtr: %d \n", *aPtr);
    return 0;
}
```

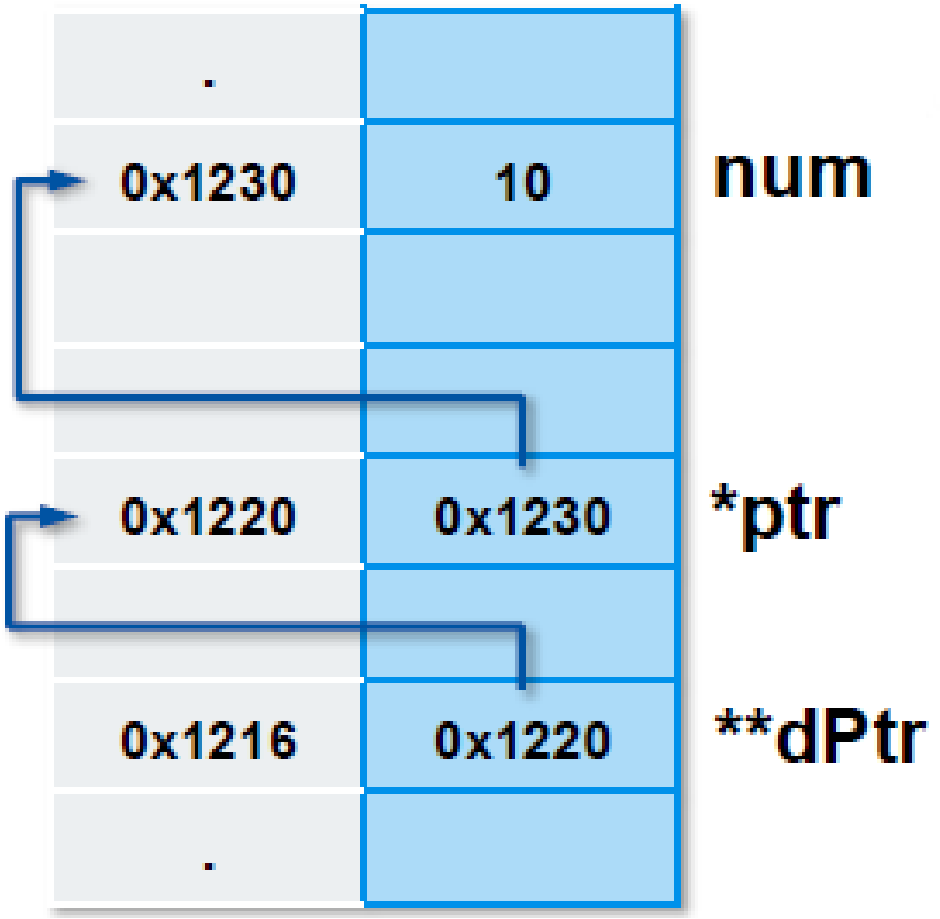
Saída no terminal:

```
Conteúdo de a: 10
Conteúdo de aPtr: 0x16b21b038
Endereço de a: 0x16b21b038
Endereço de aPtr: 0x16b21b030
Valor apontado por aPtr: 10
```

Ponteiro de Ponteiro

Ponteiro para inteiro
(*pointer to integer*)

Ponteiro para inteiro
(*pointer to integer*)



Ponteiro de Ponteiro

- Exemplo em linguagem C: Ponteiro de ponteiro

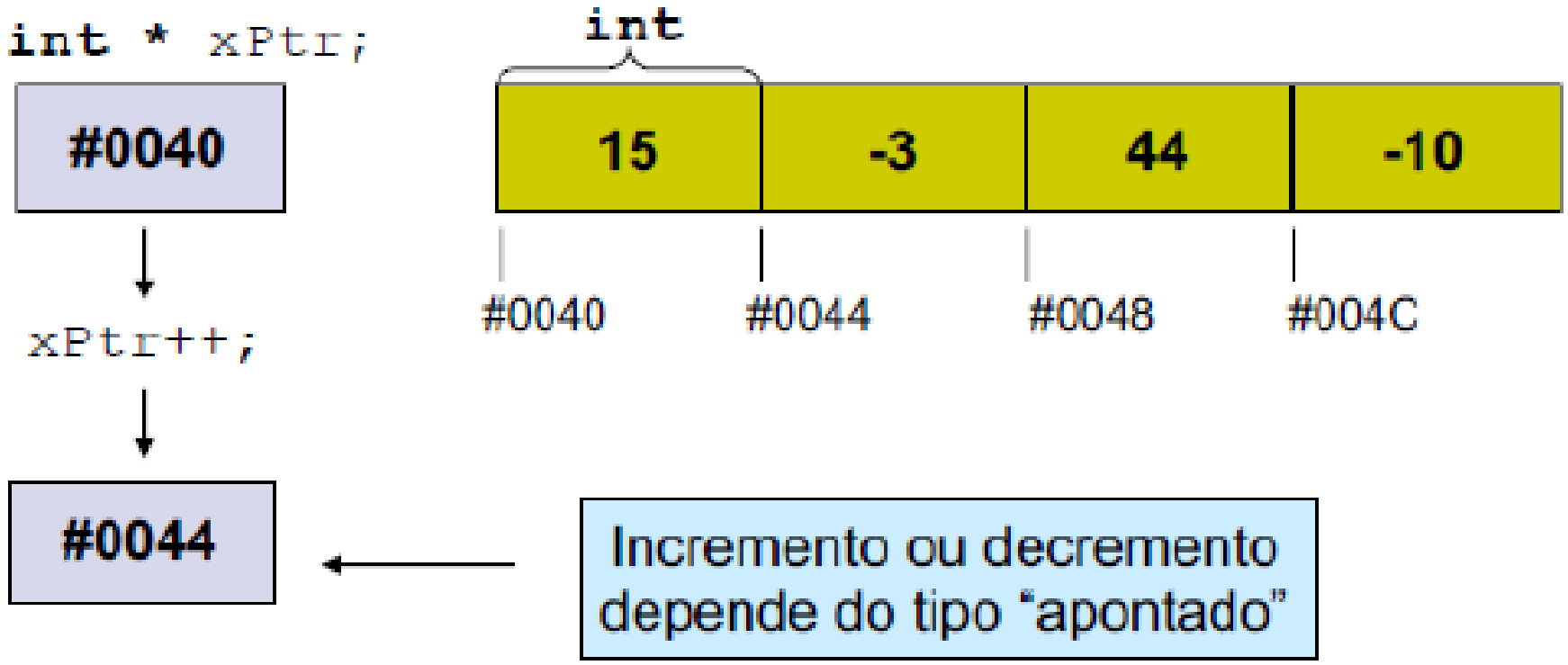
```c
#include <stdio.h>
int main(void) {
    double x = 10.0; // Declaração e inicialização de "x"
    double *xPtr;    // Declaração do ponteiro para float
    xPtr = &x;       // Atribuição do endereço de "x" para "xPtr"
    double **ptrXPtr;
    ptrXPtr = &xPtr;
    printf("Conteúdo de x: %f \n", x);
    printf("Conteúdo de xPtr: %p \n", xPtr);
    printf("Endereço de x: %p \n", &x);
    printf("Endereço de xPtr: %p \n", &xPtr);
    printf("Valor apontado por xPtr: %f \n", *xPtr);
    printf("Conteúdo de ptrXPtr: %p \n", *ptrXPtr);
    printf("Endereço de ptrXPtr: %p \n", &ptrXPtr);
    printf("Valor apontado por ptrXPtr: %p \n", *ptrXPtr);
    printf("Valor apontado pelo conteúdo de ptrXPtr: %f \n", **ptrXPtr);
    return 0;
}
```

Saída no terminal:

```
Conteúdo de x: 10.000000
Conteúdo de xPtr: 0x16faef030
Endereço de x: 0x16faef030
Endereço de xPtr: 0x16faef028
Valor apontado por xPtr: 10.000000
Conteúdo de ptrXPtr: 0x16faef030
Endereço de ptrXPtr: 0x16faef020
Valor apontado por ptrXPtr: 0x16faef030
Valor apontado pelo conteúdo de ptrXPtr: 10.000000
```

Aritmética de Ponteiros

- Um ponteiro pode ser incrementado (++) ou decrementado (--)



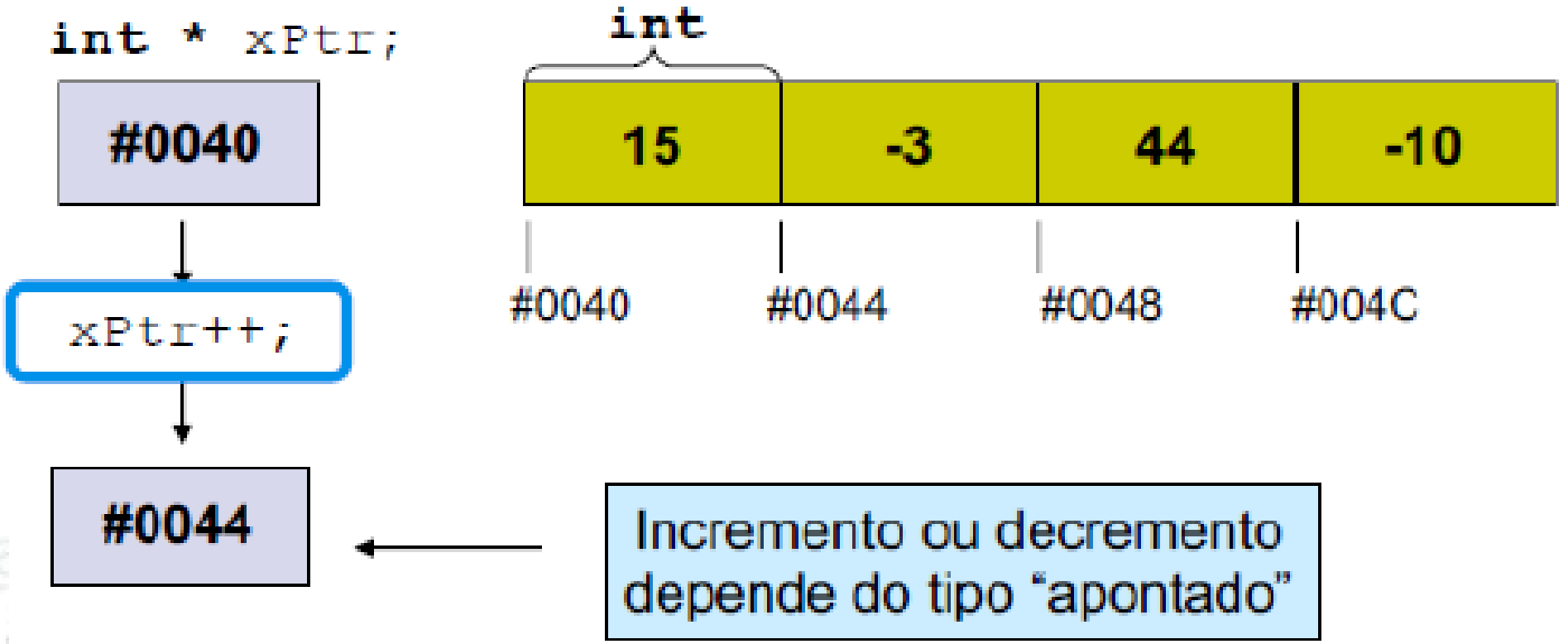
Aritmética de Ponteiros

- Relembrando o tamanho das variáveis

Tipo	Tamanho	Valores Possíveis
bool	1 byte	true e false
byte	1 byte	0 a 255
sbyte	1 byte	-128 a 127
short	2 bytes	-32768 a 32767
ushort	2 bytes	0 a 65535
int	4 bytes	-2147483648 a 2147483647
uint	4 bytes	0 to 4294967295
long	8 bytes	–9223372036854775808L to 9223372036854775807L
ulong	8 bytes	0 a 18446744073709551615
float	4 bytes	Números até 10 elevado a 38. Exemplo: 10.0f, 12.5f
double	8 bytes	Números até 10 elevado a 308. Exemplo: 10.0, 12.33
decimal	16 bytes	números com até 28 casas decimais. Exemplo 10.991m, 33.333m
char	2 bytes	Caracteres delimitados por aspas simples. Exemplo: 'a', 'ç', 'o'

Aritmética de Ponteiros

- Um ponteiro pode ser incrementado (++) ou decrementado (--)



Aritmética de Ponteiros

- Exemplo:

```
1   #include <stdio.h>
2   int main(void) {
3       int x, y, *p = NULL;
4       y = 1;
5       p = &y;
6       x = *p;
7       x = 4;
8       (*p)++;
9       --x;
10      (*p) += x;
11      return 0;
12  }
```

É uma boa prática inicializar o ponteiro com NULL. Deste modo ele começa apontando para nulo! É melhor!

x	y	p

Ponteiros e Vetores

- Na verdade, todo vetor é um ponteiro!
- Exemplo:

```c
#include <stdio.h>
int main(void) {
    int vetor[5];
    printf("%p \n", vetor);
    printf("%p \n", &vetor[0]);
    printf("%p \n", &vetor[1]);
    printf("%p \n", &vetor[2]);
    printf("%p \n", &vetor[3]);
    printf("%p \n", &vetor[4]);
    return 0;
}
```

Saída no terminal:

```
0x16d2db024
0x16d2db024
0x16d2db028
0x16d2db02c
0x16d2db030
0x16d2db034
```

Ponteiros e Vetores

- Ao declarar um vetor, uma área sequencial da memória é alocada:

```
1   #include <stdio.h>
2   int main(void) {
3       int vetor[10];
4       return 0;
5   }
```

Endereços	Valores
...	
0x1224	4
0x1220	7
0x121C	4
0x1018	3
0x1014	4
0x1010	6
0x100C	1
0x1008	2
0x1004	3
0x1000	2

Ponteiros e Vetores

- Ao declarar um vetor, uma área sequencial da memória é alocada:

```c
#include <stdio.h>
int main(void) {
    int vetor[10];
    printf("%d \n", vetor[0]);
    printf("%p \n", vetor);
    return 0;
}
```

Imprime 2

Imprime 1000

...	
0x1224	4
0x1220	7
0x121C	4
0x1018	3
0x1014	4
0x1010	6
0x100C	1
0x1008	2
0x1004	3
0x1000	2
Endereços	Valores

Ponteiros e Vetores

- Exemplo com aritmética de ponteiros:

```c
#include <stdio.h>
int main(void) {
    int vetor[5];
    int *pVetor;
    pVetor = vetor;
    vetor[0] = 10;
    vetor[1] = 20;
    vetor[2] = 30;
    vetor[3] = 40;
    vetor[4] = 50;
    printf("%p \n", pVetor);
    pVetor++;
    printf("%p \n", pVetor);
    return 0;
}
```

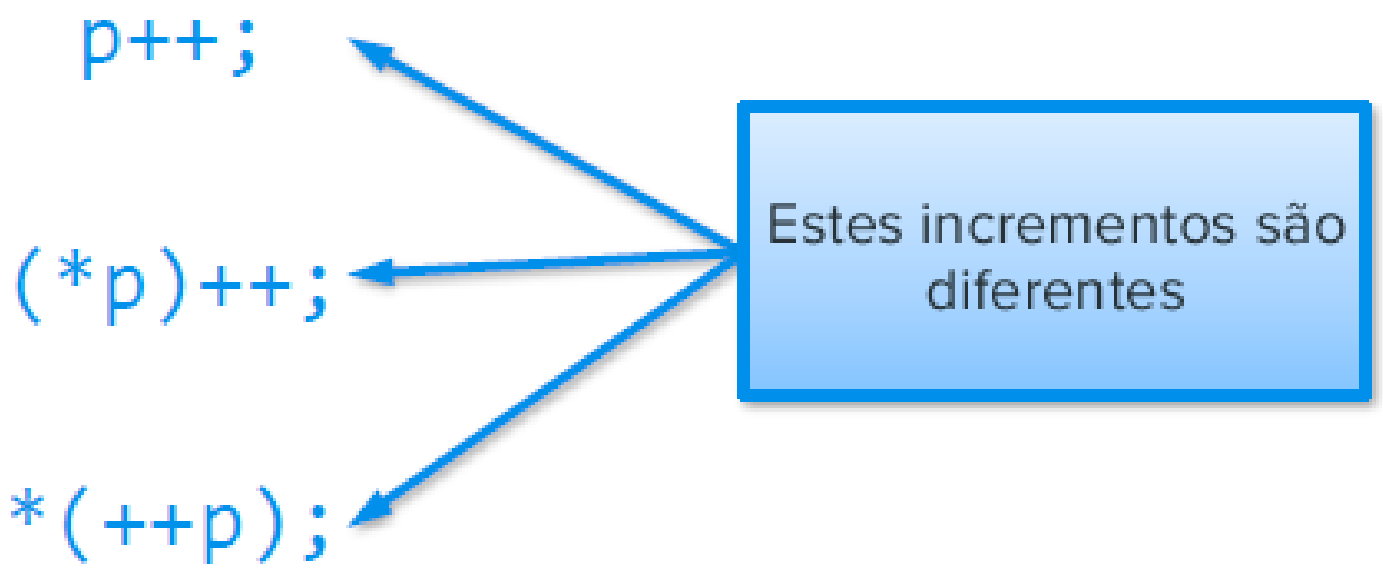
Saída no terminal:

```
0x16b3bf024
0x16b3bf028

Process finished with exit code 0
```

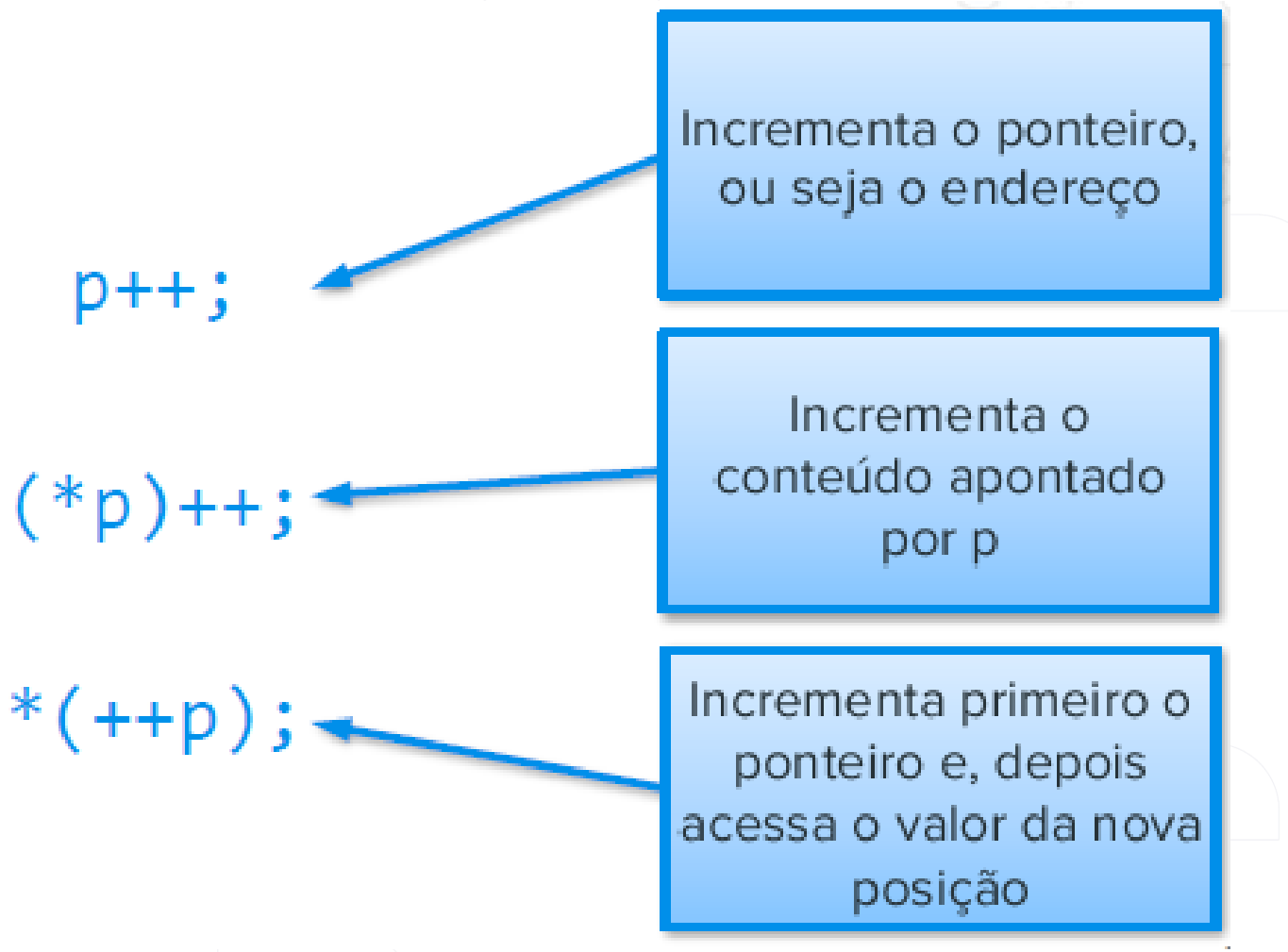
Aritmética de Ponteiros

- Cuidado:



Aritmética de Ponteiros

- Cuidado:



Aritmética de Ponteiros

- Exemplo:

```c
#include <stdio.h>
int main(void) {
    int index, vetor[5];
    int *pVetor;
    pVetor = vetor;
    for(index = 0; index < 5; index++) {
        vetor[index] = index;
    }
    printf("%d \n", (*pVetor)++ );
    printf("%d \n", *pVetor );
    pVetor++;
    printf("%d \n", (*pVetor)++ );
    printf("%d \n", *(++pVetor) );
    printf("%d \n", *(++pVetor) );
    printf("%d \n", *(++pVetor) );
    printf("%d \n", *(++pVetor) );
    return 0;
}
```

Pós-incremento do valor:
- Primeiro lê o valor apontado por pVetor (0).
- O printf exibe o valor 0.
- Depois da exibição, o conteúdo daquela memória é incrementado: vetor[0] ← 1.

O ponteiro "anda" para a próxima casa da memória. Agora ele aponta para vetor[1].

Saída: Valor Indefinido (Lixo), o ponteiro avançou para uma posição além do limite do vetor (índice 5).

```
0
1
1
2
3
4
-175177638
```

Aritmética de Ponteiros

- Exemplo:

```c
#include <stdio.h>
int main(void) {
    int index, vetor[10], *pVetor;
    pVetor = vetor;
    for(index = 0; index < 10; index++) {
        vetor[index] = 0;
    }
    for(index = 0; index < 10; index+=2) {
        *(pVetor+index) = 6;
    }
    for(index = 1; index < 10; index+=2) {
        *(pVetor+index) = 4;
    }
    printf("Vetor [ ");
    for(index = 0; index < 10; index++) {
        printf("%d ", *(pVetor+index) );
    }
    printf("]\n");
    return 0;
}
```

Exercícios

1. Escreva um programa que declare um inteiro, um real, um char, e ponteiros para inteiro, real, e char. Associe as variáveis aos ponteiros (use &). Modifique os valores de cada variável usando os ponteiros. Imprima os valores das variáveis antes e após as modificações.

2. Faça um programa para somar dois números reais utilizando ponteiros.

3. Escreva um programa que contenha duas variáveis inteiras. Leia o valor dessas variáveis pelo teclado. Em seguida, compare seus endereços e exiba o conteúdo do maior endereço.

Obrigada pela sua participação, nos vemos na próxima aula!

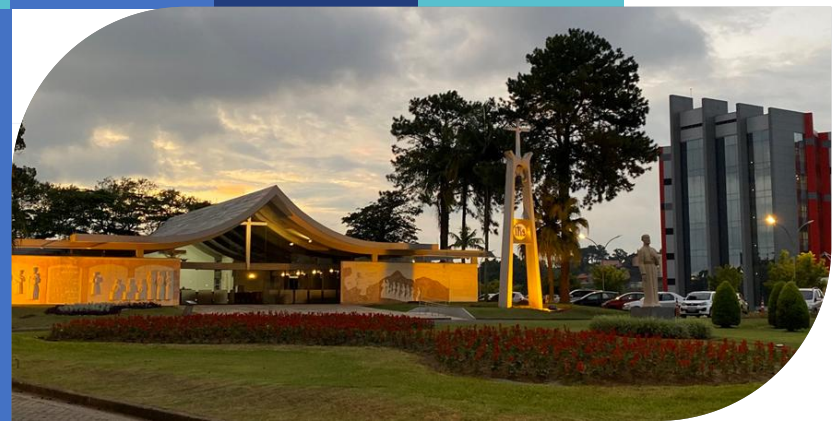
:)

JESUÍTAS BRASIL









FEI edu.br

JESUÍTAS BRASIL